Report: Optimising NYC Taxi Operations

Include your visualizations, analysis, results, insights, and outcomes. Explain your methodology and approach to the tasks. Add your conclusions to the sections.

1. Data Preparation

1.1. Loading the dataset

1.1.1. Sample the data and combine the files

As the data is huge, just keeping 1% of sample data instead of 5%

Total Combined Rows: **379268**

Here the logic used to sample 1% of data on hourly basis and stored combined data as parquet format.

- Loop through each parquet file present in `file_list`
- Read the parquet file into a pandas dataframe
- Filter dataframe records to keep only rows matching the year-month from the file name
- Extract all unique days from `tpep_pickup_datetime`
- Loop through each day separately
- For each day:
  - create a daily dataframe containing only that day's records
  - Extract all unique hours from that day
  - Loop through each hour separately
  - For each hour:
    - ✓ create an hourly dataframe containing only that hour’s records
    - ✓ Randomly sample 1% of records from that hourly dataframe
    - ✓ Append sampled hourly data into `combined_df`

2. Data Cleaning

2.1. Fixing Columns

2.1.1. Fix the index

Once all the unnecessary columns are dropped, this is how we would fix the index.

```
df.reset_index(drop=True)
```

2.1.2. Combine the two airport_fee columns

We have following fields with percentage of missing values:

airport_fee 92.176245

Airport_fee 11.236118

```
# Combine the two airport fee columns
df['Airport_fee'] = df['Airport_fee'].fillna(df['airport_fee'])
# Also if Airport_fee still has NaN, fill them with 0
df['Airport_fee'] = df['Airport_fee'].fillna(0)

# Drop other airport_fee column
df.drop(columns=['airport_fee'], inplace=True)
```

2.2. Handling Missing Values

2.2.1. Find the proportion of missing values in each column

Column Name	Missing %
VendorID	0
tpep_pickup_datetime	0
tpep_dropoff_datetime	0
passenger_count	3.412363
trip_distance	0
RatecodeID	3.412363
store_and_fwd_flag	3.412363
PULocationID	0
DOLocationID	0
payment_type	0
fare_amount	0
extra	0
mta_tax	0
tip_amount	0
tolls_amount	0
improvement_surcharge	0
total_amount	0

congestion_surcharge	3.412363
airport_fee	92.176245
Airport_fee	11.236118

2.2.2. Handling missing values in passenger_count

We have found quite a lot NaN values in 'passenger_count' column and replaced them with 0 as below:

```
# There are almost 12942 records with missing values
print(df['passenger_count'].isna().sum())

# Fill the missing values (NaN) with 0
df['passenger_count'] = df['passenger_count'].fillna(0)
```

2.2.3. Handle missing values in RatecodeID

We have found quite a lot NaN values in 'RatecodeID' column and replaced them with 1 as below:

```
# There are almost 12942 records with missing values

print(df['RatecodeID'].isna().sum())

# Fill the missing values (NaN) with 0

df['RatecodeID'] = df['RatecodeID'].fillna(1)
```

Where 1= Standard rate for RatecodeID

2.2.4. Impute NaN in congestion_surcharge

We have found quite a lot NaN values in '**congestion_surcharge**' column and replaced them with 0 as below:

```
# There are almost 12942 records with missing values
print(df['congestion_surcharge'].isna().sum())


# Fill the missing values (NaN) with 0

df['congestion_surcharge'] = df['congestion_surcharge'].fillna(0)
```

2.3. Handling Outliers and Standardising Values

2.3.1. Check outliers in payment type, trip distance and tip amount columns

payment type:

```
# Valid values for payment_type: 1= Credit card, 2= Cash, 3= No charge, 4= Dispute, 5= Unknown, 6= Voided trip

df['payment_type'].unique()

# Keep only valid payment_type values from 1 to 6

df = df[ df['payment_type'].isin([1, 2, 3, 4, 5, 6]) ]
```

trip distance:

```
# Assumption here:

# Trips with extremely large distance (>250 miles) # but unusually low fare (<100) seems invalid.

# find out trips having trip_distance > 250

df['trip_distance'] > 250

# Before removing these records, let's also check fare_amount along with trip_distance

print(len(df[ (df['trip_distance'] > 250)  & (df['fare_amount'] < 100) ]))

# Keep only valid trip_distance values

df = df[ ~(df['trip_distance'] > 250)  & (df['fare_amount'] < 100) ]

# Entries where trip_distance and fare_amount are 0 but the pickup and dropoff zones are different

diff_zones_df = df[ (df['trip_distance'] == 0) &  (df['fare_amount'] == 0)

& (df['PULocationID'] != df['DOLocationID']) ]
```

tip amount:

```
# Check for tip_amount is negative or very high and more than fare_amount

print(df[ (df['tip_amount'] < 0) | (df['tip_amount'] > 100) & (df['fare_amount'] < df['tip_amount']) ])
```

```
# For negative cap it to 0 and for higher value than 100 keep to 10% of fare amount

df.loc[ (df['tip_amount'] > 100) & (df['fare_amount'] < df['tip_amount']), 'tip_amount'] = df['fare_amount']*0.1
```

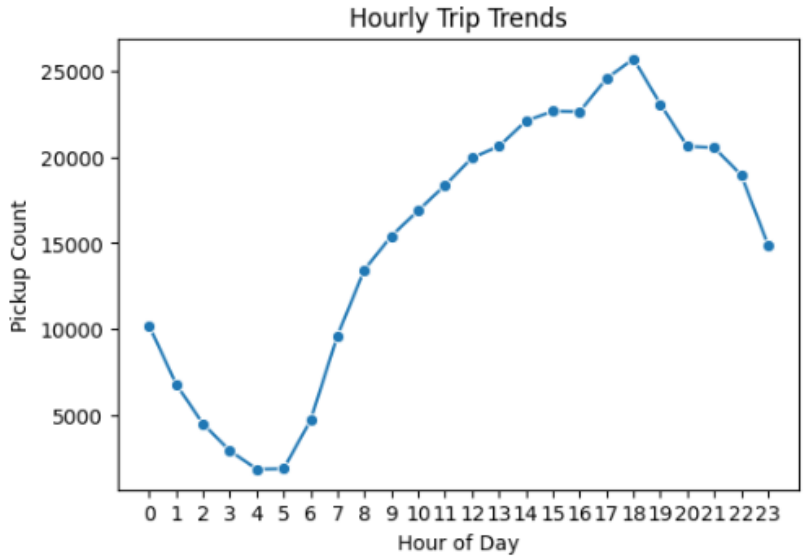
3. Exploratory Data Analysis

3.1. General EDA: Finding Patterns and Trends

3.1.1. Classify variables into categorical and numerical

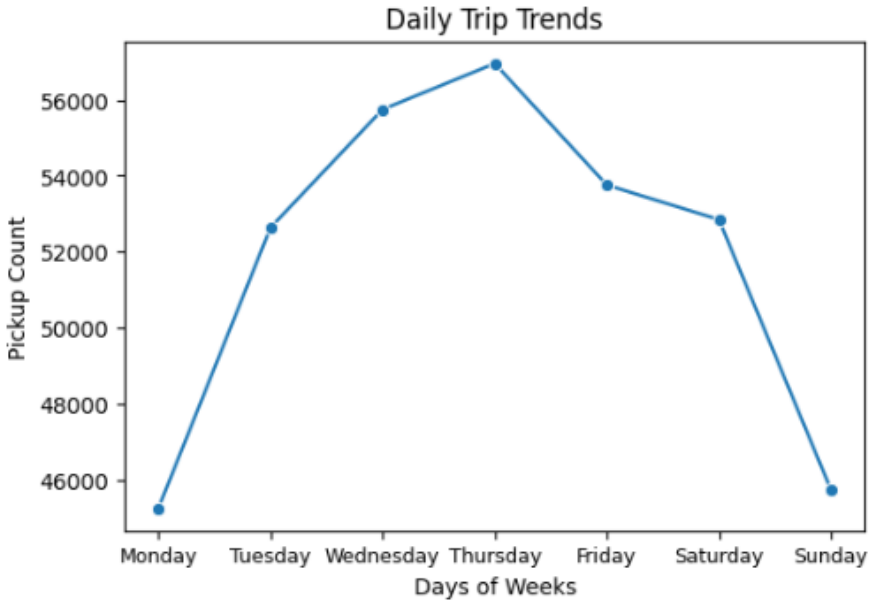
Column Name	Classification	Remarks
VendorID	Categorical	It has got only 2 unique values
tpep_pickup_datetime	Datetime type	Neither Numerical nor Categorical
tpep_dropoff_datetime	Datetime type	Neither Numerical nor Categorical
passenger_count	Numerical	
trip_distance	Numerical	
RatecodeID	Categorical	It has got only 6 unique values
store_and_fwd_flag	Categorical	Y or N
PULocationID	Categorical	Though values are Numerical but only 250~260 distinct values
DOLocationID	Categorical	Though values are Numerical but only 250~260 distinct values
payment_type	Categorical	It has got only 6 unique values
fare_amount	Numerical	
extra	Numerical	
mta_tax	Numerical	
tip_amount	Numerical	
tolls_amount	Numerical	
improvement_surcharge	Numerical	
total_amount	Numerical	
congestion_surcharge	Numerical	
Airport_fee	Numerical	

3.1.2. Analyse the distribution of taxi pickups by hours, days of the week, and months



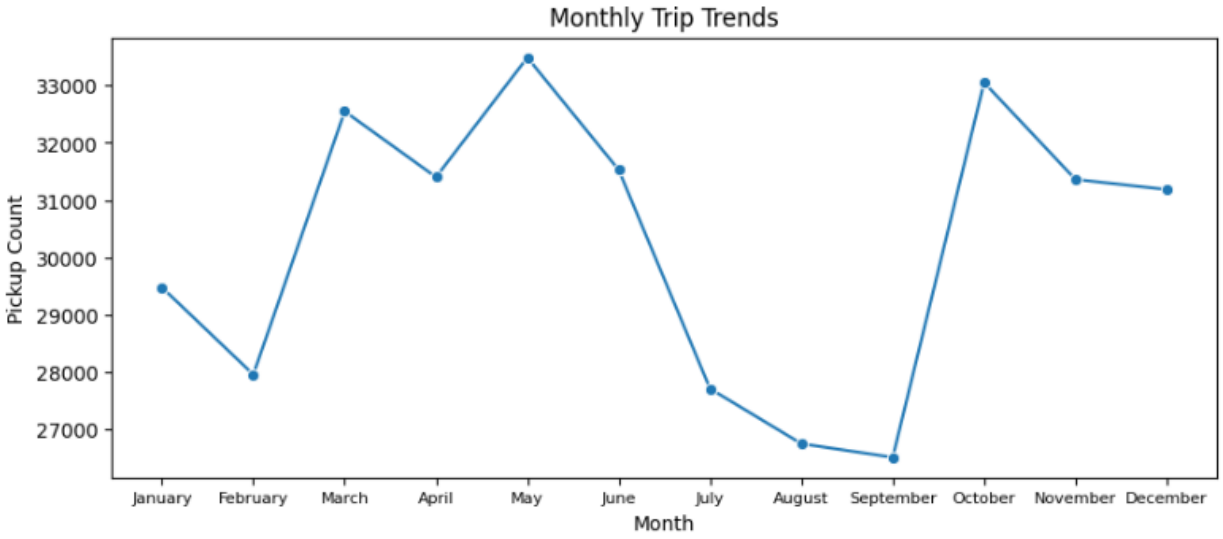
Hourly Trip Findings:

After 6 AM, trips increase rapidly. Trips decline sharply after midnight and reach the minimum around: 4 AM to 5 AM



Daily Trip Findings:

Monday has the lowest trip demand. Thursday is the busiest day and then trip demand drops slightly from Thursday.



Monthly Trip Findings:

Demand rises strongly from February to May wherein May showing the highest pickup count and then demand drops after June. September shows the lowest demand and October shows a sharp increase again.

3.1.3. Filter out the zero/negative values in fares, distance and tips

For the following parameters

lst = ['fare_amount', 'tip_amount', 'total_amount', 'trip_distance']

Output:

Negative values found in total_amount: 15

Zero values found in fare_amount: 115

Zero values found in tip_amount: 79136

Zero values found in total_amount: 45

Zero values found in trip_distance: 4145

Create a df with non zero entries for the selected parameters.

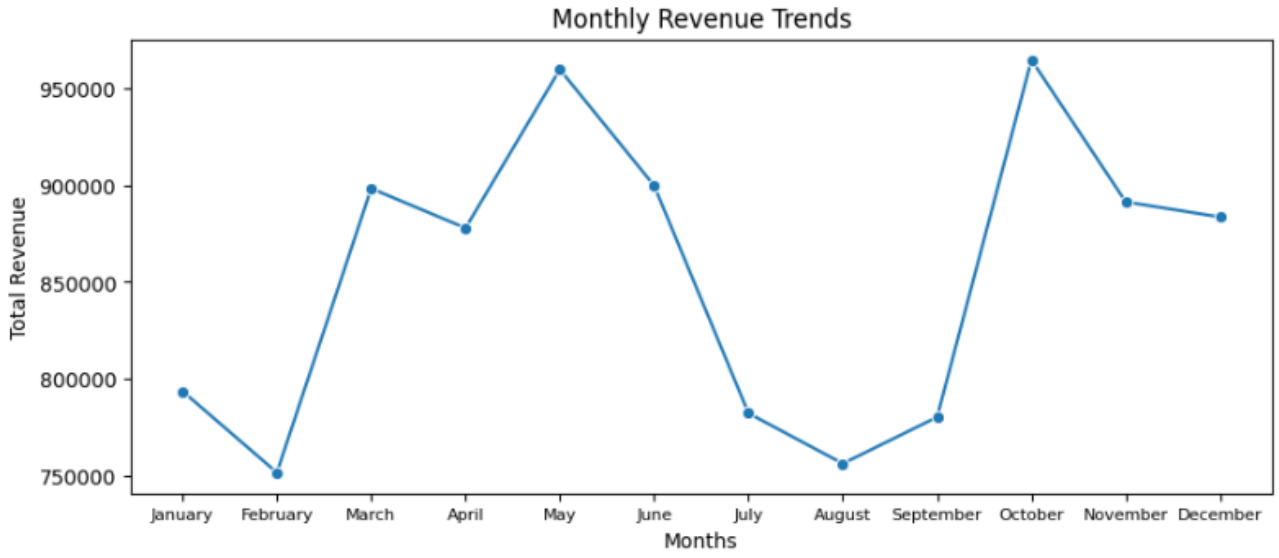
non_zero_df = df[(df[cols] != 0).any(axis=1)].copy()

Output:

Actual Dataframe count with zero entries: 362948

Copy of Dataframe count with non-zero entries: 362916

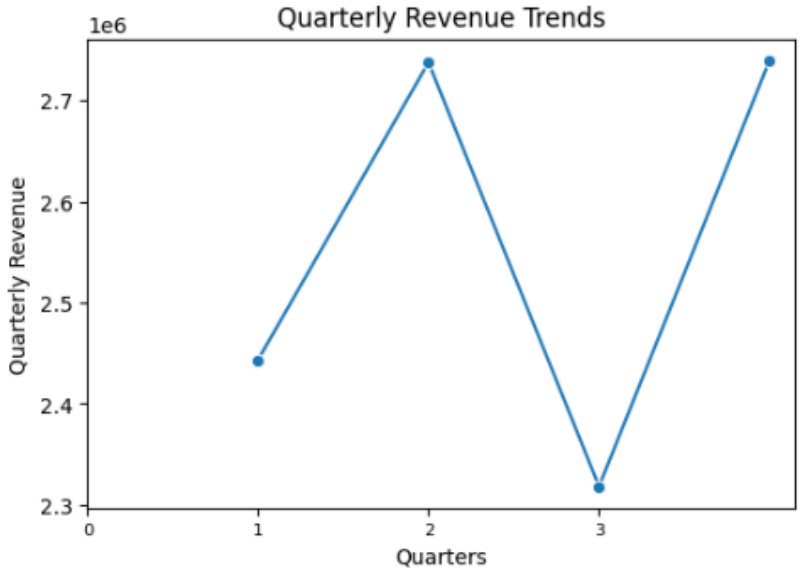
3.1.4. Analyse the monthly revenue trends



Monthly Revenue Findings:

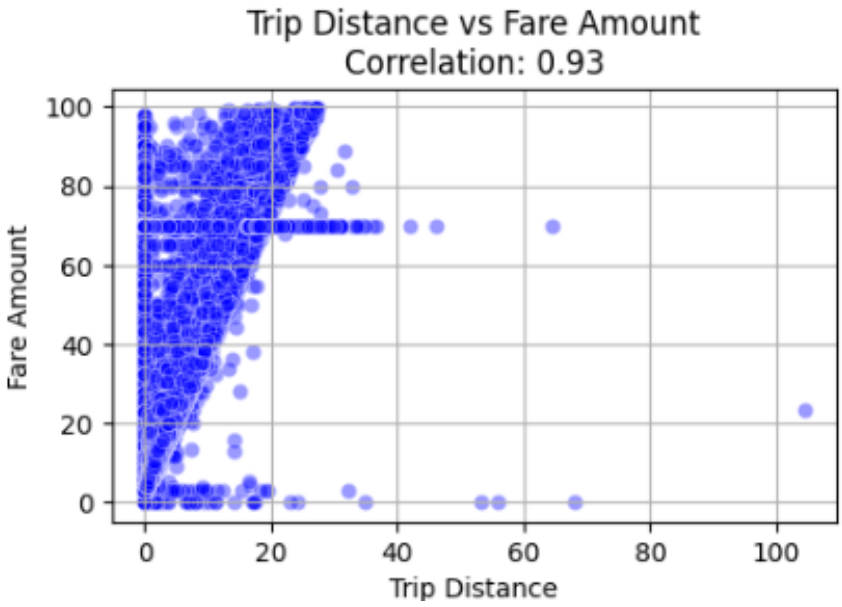
As the demand raises so the revenue rises strongly from February to May wherein May showing the highest revenue count and then it drops after June. September shows the lowest revenue and October shows a sharp increase again.

3.1.5. Find the proportion of each quarter's revenue in the yearly revenue



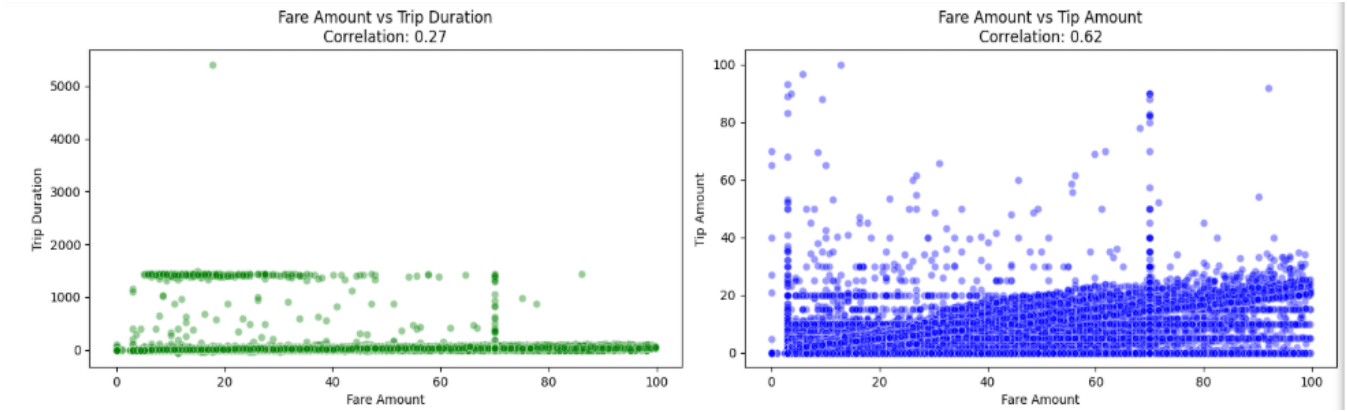
Quarterly Revenue Findings: The revenue rises from quarter 1 to quarter 2 and shows sudden drop of revenue in quarter 3 and then rises sharply in quarter 4.

3.1.6. Analyse and visualise the relationship between distance and fare



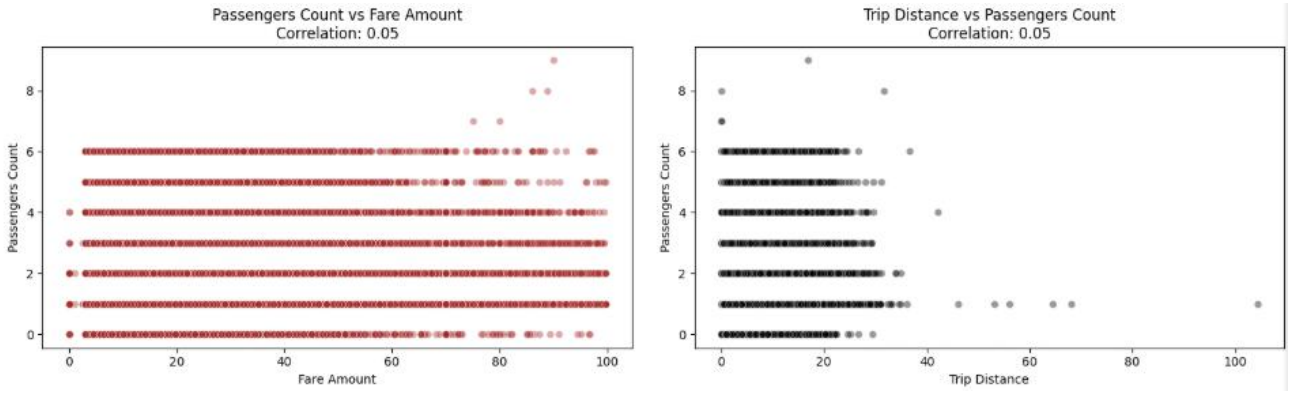
Analysis shows a very strong positive correlation. As trip distance increases, fare amount generally increases proportionally. Most trips are concentrated around: 0-20 miles.

3.1.7. Analyse the relationship between fare/tips and trips/passengers



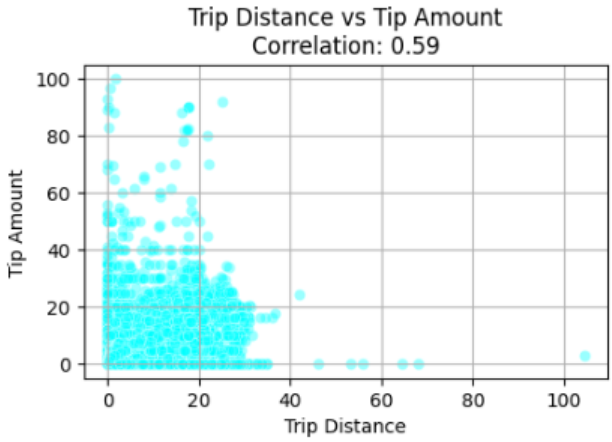
In the 1st graph analysis shows Correlation is low (0.27), which means fare is not strongly dependent on trip duration alone. Many trips have low duration but varying fares, suggesting distance contributes more to pricing.

Above in the 2nd graph analysis shows Moderate positive correlation (0.62) which means higher fares generally receive higher tips. Most tips increase proportionally with fare amount.



In the 3rd graph analysis shows that correlation is extremely low (0.05), indicating passenger count has almost no impact on fare amount. Fare is usually determined by distance, duration, and other charges rather than number of passengers.

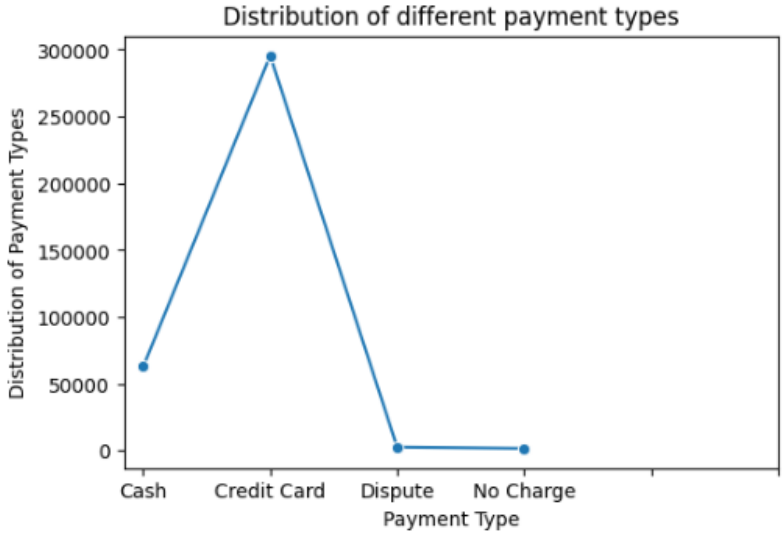
Similarly in the 4th graph analysis shows that correlation is very weak (**0.05**), meaning trip distance is independent of passenger count.



In this last graph, analysis shows moderate positive correlation (**0.59**) indicates longer trips generally tend to receive higher tips. Most trips are concentrated within 0–25 miles and tips below 20. Several high-tip outliers exist even for shorter trips.

3.1.8. Analyse the distribution of different payment types

	payment_type_name	dist_count
0	Cash	62953
1	Credit Card	295555
2	Dispute	2703
3	No Charge	1737

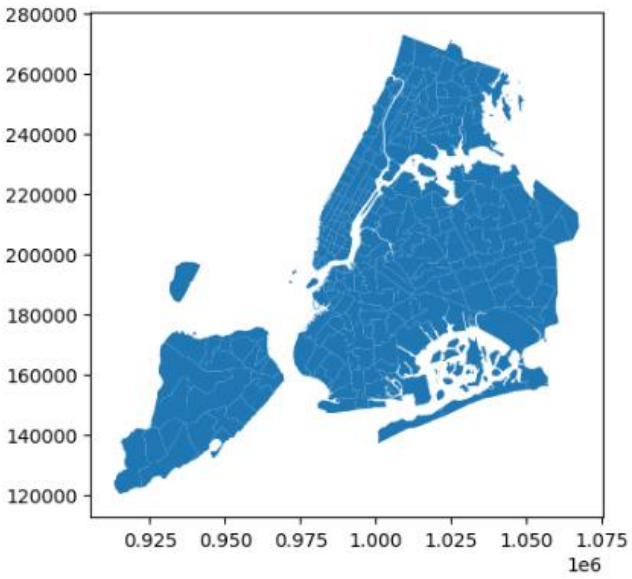


In the above graph, analysis shows that Credit Card is the dominant payment method with a high number of trips (~295K), indicating strong preference for card-based transactions. “Dispute” and “No Charge” categories are quite rare, suggesting most transactions are completed successfully without issues.

3.1.9. Load the taxi zones shapefile and display it

```
zones = gpd.read_file('trip_zones/taxi_zones.shp')

zones.head()
```

	OBJECTID	Shape_Leng	Shape_Area	zone	LocationID	borough	geometry
0	1	0.116357	0.000782	Newark Airport	1	EWR	POLYGON ((933100.918 192536.086, 933091.011 19...
1	2	0.433470	0.004866	Jamaica Bay	2	Queens	MULTIPOLYGON (((1033269.244 172126.008, 103343...
2	3	0.084341	0.000314	Allerton/Pelham Gardens	3	Bronx	POLYGON ((1026308.77 256767.698, 1026495.593 2...
3	4	0.043567	0.000112	Alphabet City	4	Manhattan	POLYGON ((992073.467 203714.076, 992068.667 20...
4	5	0.092146	0.000498	Arden Heights	5	Staten Island	POLYGON ((935843.31 144283.336, 936046.565 144...



3.1.10. Merge the zone data with trips data

```
merged_df = non_zero_df.merge(
    zones,
    left_on='PULocationID',
    right_on='LocationID',
    how='inner'
)
```

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID
0	2	2023-01-01 00:07:18	2023-01-01 00:23:15	1.0	7.74	1.0	N	138	256
1	2	2023-01-01 00:16:41	2023-01-01 00:21:46	2.0	1.24	1.0	N	161	237
2	2	2023-01-01 00:14:03	2023-01-01 00:24:36	3.0	1.44	1.0	N	237	141
3	2	2023-01-01 00:24:30	2023-01-01 00:29:55	1.0	0.54	1.0	N	143	142
4	1	2023-01-01 00:42:56	2023-01-01 01:16:33	2.0	7.10	1.0	N	246	37

3.1.11. Find the number of trips for each zone/location ID

```
trip_counts_by_loc = (
merged_df.groupby('LocationID').size().reset_index(name='trip_count') )
```

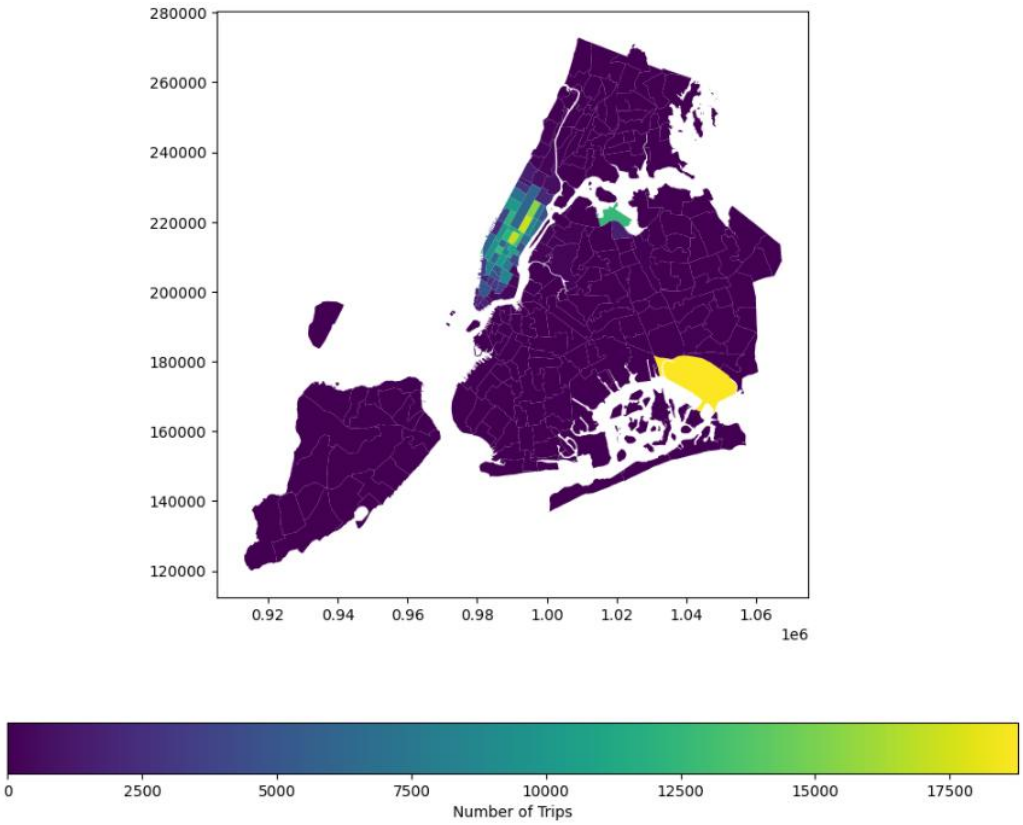
	LocationID	trip_count
0	1	23
1	3	2
2	4	353
3	6	4
4	7	148

3.1.12. Add the number of trips for each zone to the zones dataframe

```
zones = zones.merge( trip_counts_by_loc[ ['LocationID', 'trip_count'] ],
on='LocationID', how='left' )

# For given location id, fill missing values with 0

zones['trip_count'] = ( zones['trip_count'].fillna(0) )
```

	OBJECTID	Shape_Leng	Shape_Area	zone	LocationID	borough	geometry	trip_count
0	1	0.116357	0.000782	Newark Airport	1	EWR	POLYGON ((933100.918 192536.086, 933091.011 19...	23.0
1	2	0.433470	0.004866	Jamaica Bay	2	Queens	MULTIPOLYGON (((1033269.244 172126.008, 103343...	0.0
2	3	0.084341	0.000314	Allerton/Pelham Gardens	3	Bronx	POLYGON ((1026308.77 256767.698, 1026495.593 2...	2.0
3	4	0.043567	0.000112	Alphabet City	4	Manhattan	POLYGON ((992073.467 203714.076, 992068.667 20...	353.0
4	5	0.092146	0.000498	Arden Heights	5	Staten Island	POLYGON ((935843.31 144283.336, 936046.565 144...	0.0

3.1.13. Plot a map of the zones showing number of trips



3.1.14. Conclude with results

- Manhattan is the area where highest number of trip volume observed.
- One isolated high-density zone (The one in yellow color) in Queens.
- Outer boroughs shows very low trip counts.

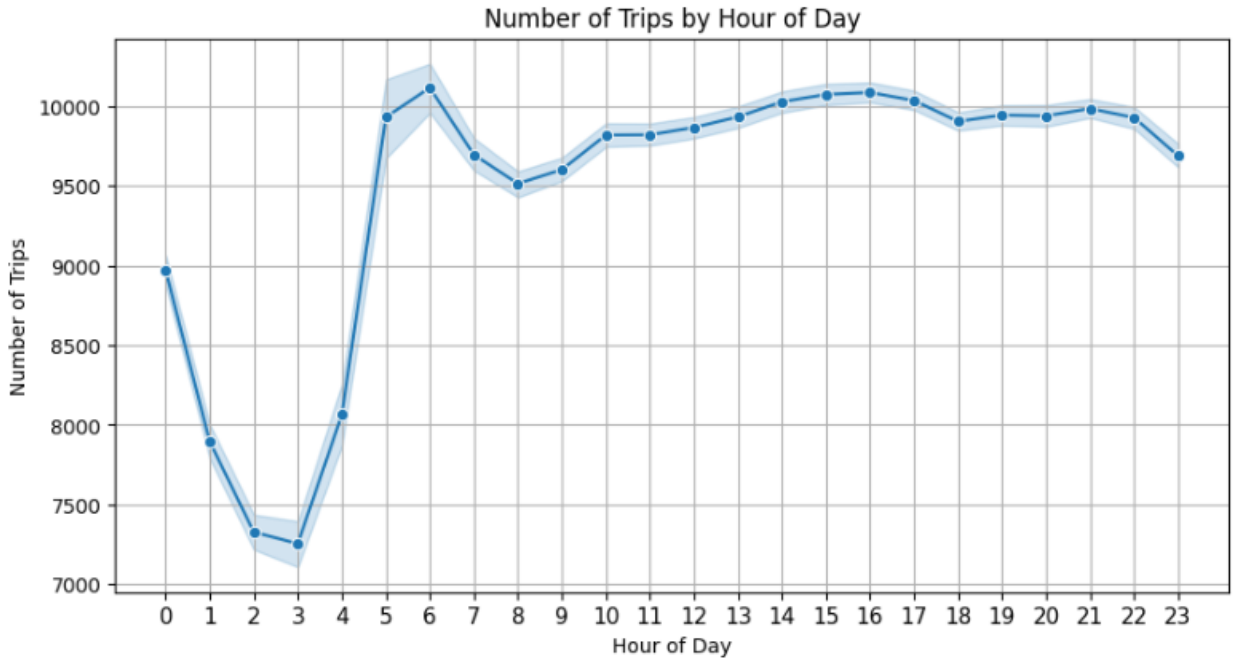
3.2. Detailed EDA: Insights and Strategies

3.2.1. Identify slow routes by comparing average speeds on different routes

	tpep_pickup_hour	PULocationID	DOLocationID	avg_trip_distance	avg_trip_duration_mins	avg_speed_per_hr
12882	0	88	144	1.780000	1425.466667	0.074923
59132	1	238	249	4.580000	1396.200000	0.196820
63184	2	255	256	0.010000	4.250000	0.141176
34887	3	148	238	6.040000	1429.983333	0.253430
50241	4	230	51	16.960000	1417.666667	0.717799
63353	5	261	14	10.530000	1117.466667	0.565386
8603	6	70	138	1.490000	1042.566667	0.085750
38225	7	161	238	2.810000	705.600000	0.238946
5581	8	50	43	1.420000	1431.333333	0.059525
31461	9	142	232	6.710000	1411.400000	0.285249
14023	10	90	186	0.661429	108.047619	0.367298
59948	11	239	170	3.080000	377.845833	0.489088
48891	12	229	45	5.770000	1430.933333	0.241940
47062	13	209	232	1.345000	721.816667	0.111801
55356	14	234	232	2.140000	1433.383333	0.089578
66527	15	264	237	0.040000	55.766667	0.043036
4714	16	48	145	4.050000	1436.333333	0.169181
63275	17	260	129	0.960000	1413.633333	0.040746
48699	18	226	145	1.200000	2709.900000	0.026569
18048	19	113	181	0.090000	35.250000	0.153191
44818	20	181	132	2.290000	1499.816667	0.091611
41735	21	164	100	0.835000	352.329167	0.142197
18264	22	113	235	0.280000	349.233333	0.048105
34028	23	148	45	0.800000	723.908333	0.066307

3.2.2. Calculate the hourly number of trips and identify the busy hours

From the following graph, analysis shows that the busiest hours are during the late afternoon and evening, around 2 PM to 5 PM, where trip counts stay at peak. A sharp rise in trips is also noted around 5 AM to 6 AM where trip count reaches the maximum.



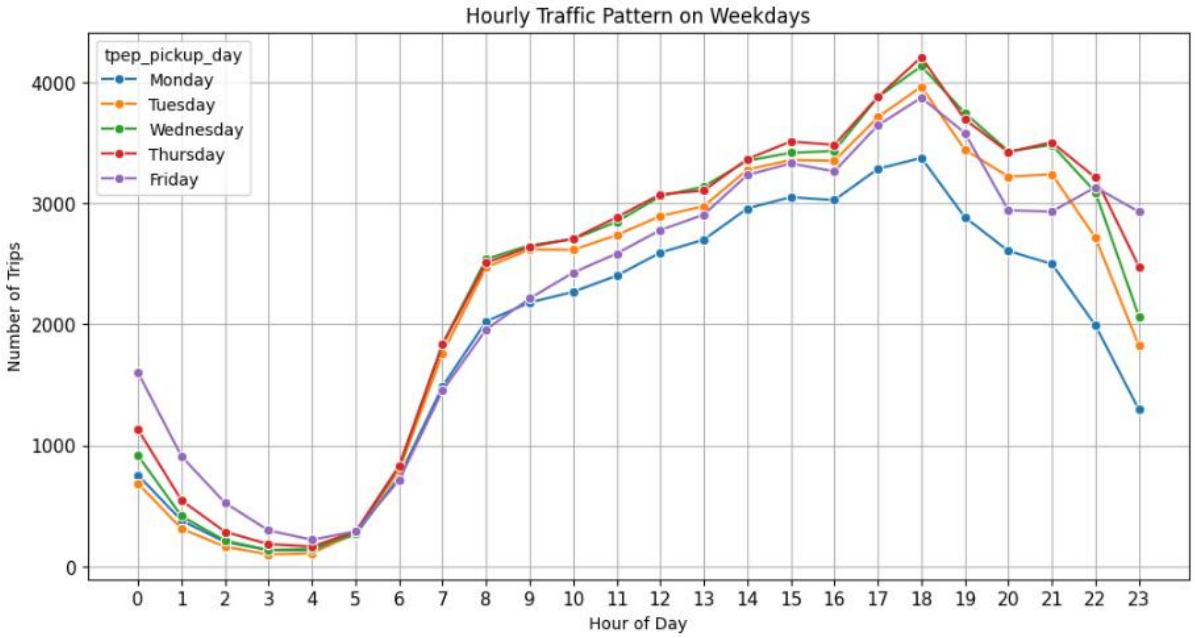
3.2.3. Scale up the number of trips from above to find the actual number of trips

tpep_pickup_hour	actual_est_count
18	2.522157e+10
17	2.442388e+10
19	2.273574e+10
15	2.262314e+10
16	2.261905e+10

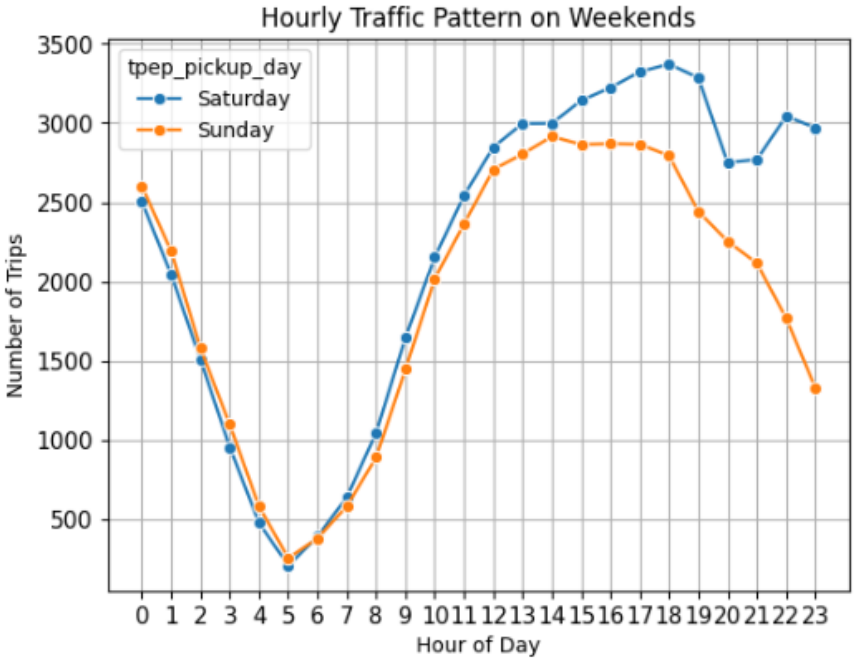
3.2.4. Compare hourly traffic on weekdays and weekends

In below graph, the analysis shows that all weekdays show a very similar traffic pattern, indicating consistent commutation. Traffic is lowest during the early morning hours (2 AM – 5 AM) and rises sharply after 6 AM. Trip counts continue increasing throughout the day and peak around 5 PM – 6

PM. Thursday appears to be the busiest weekday overall and Monday consistently has the lowest traffic volume.



In below graph, the analysis shows that Saturday peaks around 5 PM – 6 PM, suggesting strong weekend movement whereas Sunday traffic begins declining earlier in the evening. Overall on weekends, traffic drops sharply between 2 AM – 5 AM and then starts rising from 6 AM.



3.2.5. Identify the top 10 zones with high hourly pickups and drops

Top 10 zones with high hourly pickups:

	tpep_pickup_hour	PULocationID	zone	borough	pickup_count
2077	18	161	Midtown Center	Manhattan	1500
1963	17	161	Midtown Center	Manhattan	1479
1997	17	237	Upper East Side South	Manhattan	1360
2510	22	132	JFK Airport	Queens	1335
1751	15	237	Upper East Side South	Manhattan	1333
1692	15	132	JFK Airport	Queens	1330
1841	16	161	Midtown Center	Manhattan	1323
2191	19	161	Midtown Center	Manhattan	1320
1750	15	236	Upper East Side North	Manhattan	1300
2112	18	237	Upper East Side South	Manhattan	1289

Top 10 zones with high hourly drop-offs:

	tpep_dropoff_hour	DOLocationID	zone	borough	dropoff_count
38623	15	236	Upper East Side South	Manhattan	230
41955	16	236	Upper East Side South	Manhattan	230
45405	17	236	Upper East Side South	Manhattan	216
48943	18	236	Upper East Side South	Manhattan	206
28987	12	237	Upper East Side North	Manhattan	204
48995	18	237	Upper East Side North	Manhattan	197
32116	13	236	Upper East Side South	Manhattan	192
32166	13	237	Upper East Side North	Manhattan	182
38672	15	237	Upper East Side North	Manhattan	180
35308	14	236	Upper East Side South	Manhattan	180

3.2.6. Find the ratio of pickups and dropoffs in each zone

Top 10 zones with high pickups ratio

	zone	pickup_dropoff_ratio
102	JFK Airport	18774.0
198	Upper East Side South	17340.0
131	Midtown Center	17101.0
197	Upper East Side North	15511.0
132	Midtown East	13206.0
153	Penn Station/Madison Sq West	12655.0
111	LaGuardia Airport	12576.0
191	Times Sq/Theatre District	12122.0
114	Lincoln Square East	12067.0
140	Murray Hill	10808.0

 # Bottom 10 zones with high pickups ratio

	zone	pickup_dropoff_ratio
21	Bronx Park	1.0
22	Bronxdale	1.0
50	Cypress Hills	1.0
36	City Island	1.0
52	Douglaston	1.0
84	Glendale	1.0
78	Fordham South	1.0
70	Far Rockaway	1.0
82	Fresh Meadows	1.0
99	Hunts Point	1.0

3.2.7. Identify the top zones with high traffic during night hours

Top 10 pickup zones for night hours

	zone	pickup_count
74	JFK Airport	1227
83	LaGuardia Airport	795
48	East Village	727
159	West Village	665
86	Lincoln Square East	605
143	Times Sq/Theatre District	603
27	Clinton East	594
100	Midtown Center	569
117	Penn Station/Madison Sq West	546
91	Lower East Side	477

Top 10 drop-off zones for night hours

	zone	dropoff_count
72	JFK Airport	1365
81	LaGuardia Airport	772
46	East Village	742
156	West Village	688
141	Times Sq/Theatre District	679
84	Lincoln Square East	679
27	Clinton East	642
98	Midtown Center	629
115	Penn Station/Madison Sq West	552
89	Lower East Side	465

3.2.8. Find the revenue share for nighttime and daytime hours

Night hours: 11 pm to 5 pm

Day hours: 6 am to 10 pm

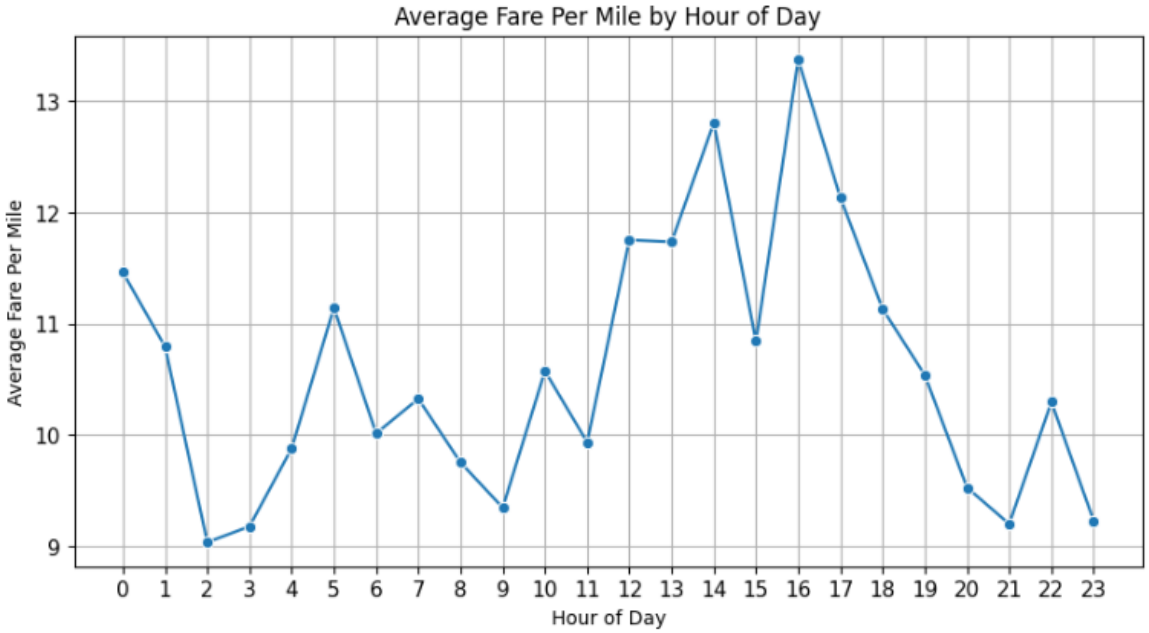
Total night revenue	1225387.28
Total day revenue	10133545.52

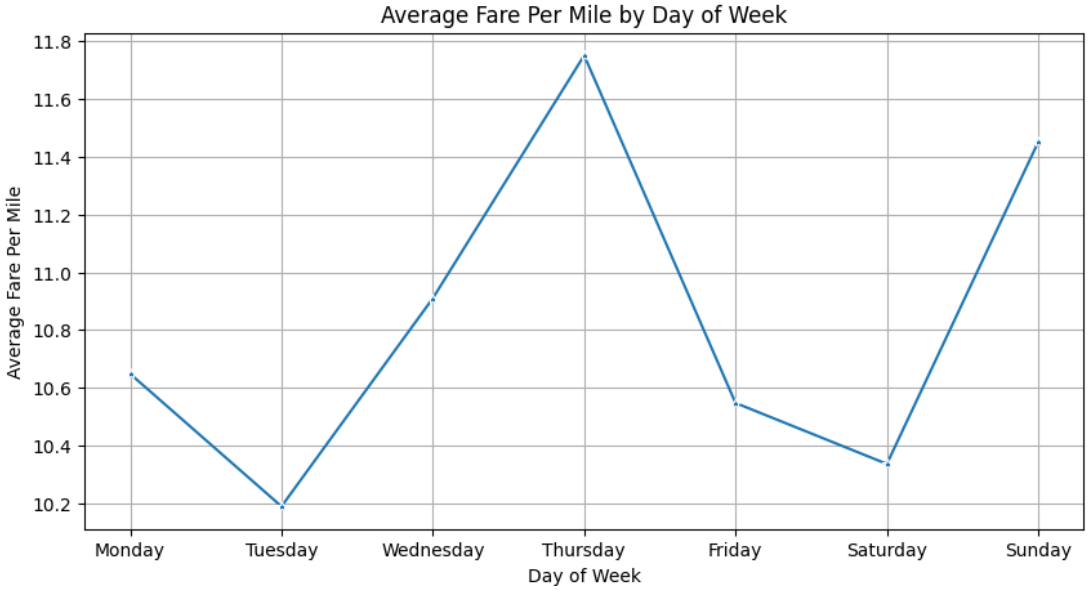
3.2.9. For the different passenger counts, find the average fare per mile per passenger

```
# Calculate fare per mile

filtered_df['fare_per_mile'] = ( filtered_df['fare_amount'] /
filtered_df['trip_distance'] )

filtered_df['fare_per_mile_per_passenger'] = ( filtered_df['fare_per_mile'] /
filtered_df['passenger_count'] )
```

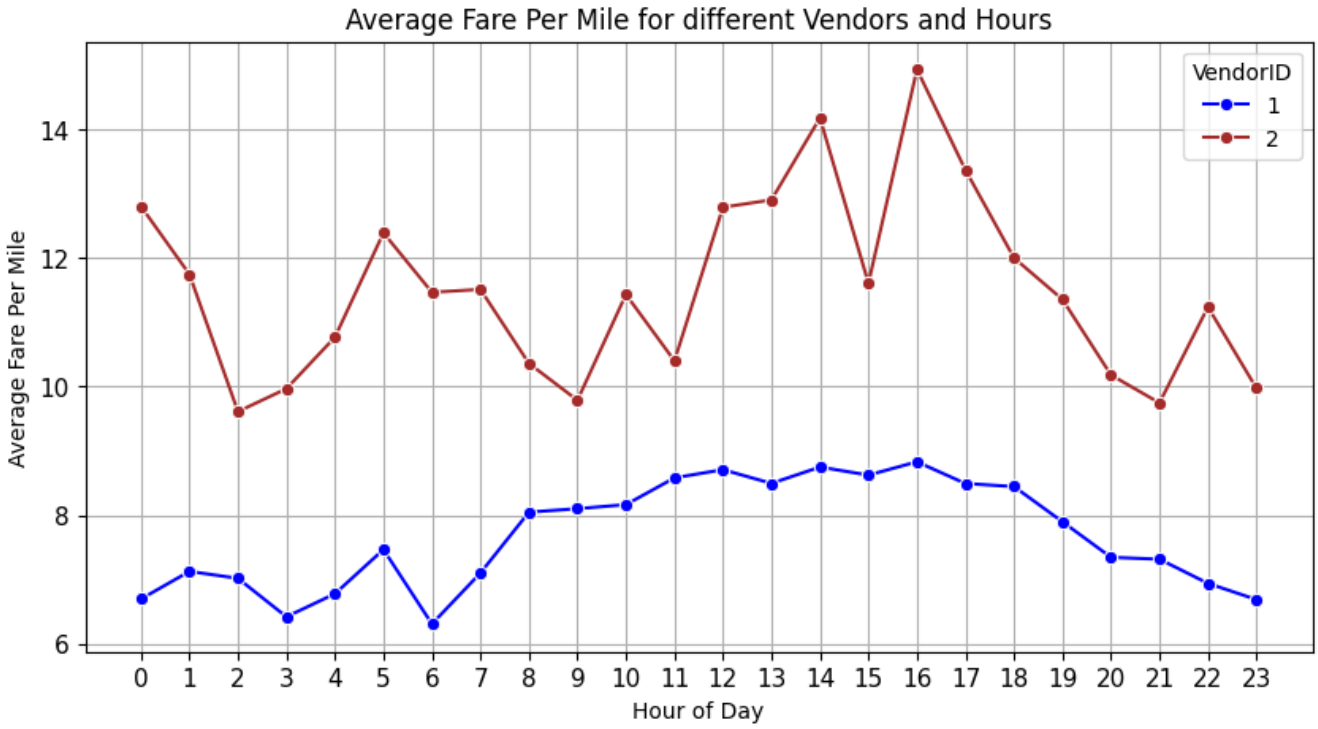
	trip_distance	passenger_count	fare_amount	fare_per_mile	fare_per_mile_per_passenger
0	7.74	1.0	32.4	4.186047	4.186047
1	1.24	2.0	7.9	6.370968	3.185484
2	1.44	3.0	11.4	7.916667	2.638889
3	0.54	1.0	6.5	12.037037	12.037037
4	7.10	2.0	34.5	4.859155	2.429577
...	...	...	...	...	...

3.2.10. Find the average fare per mile by hours of the day and by days of the week

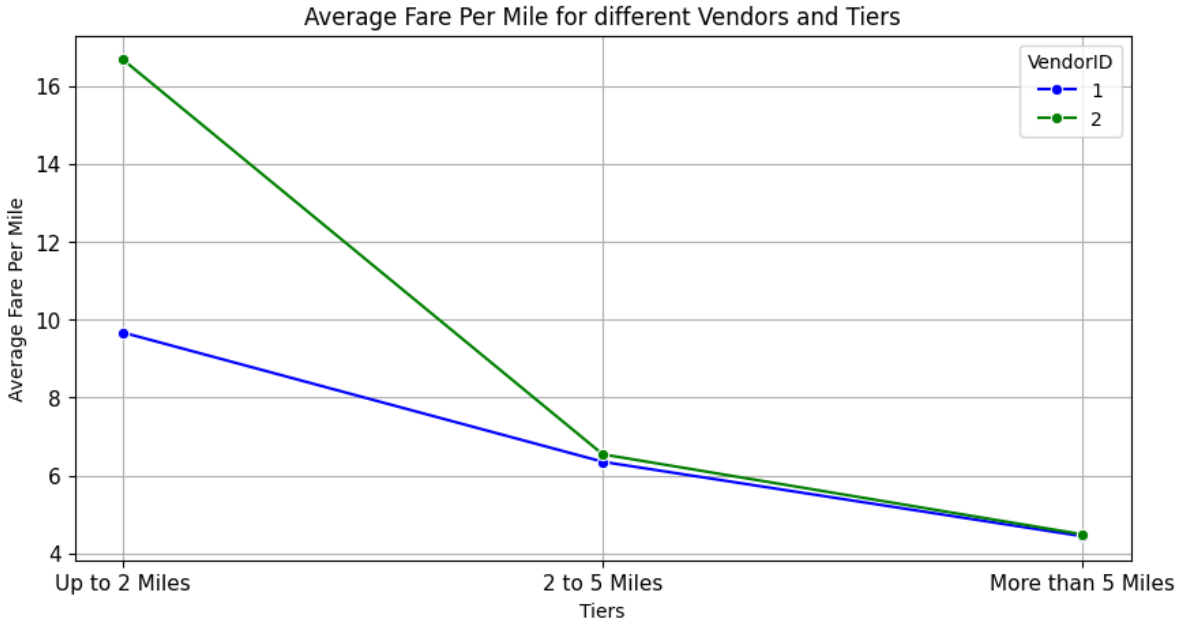




3.2.11. Analyse the average fare per mile for the different vendors

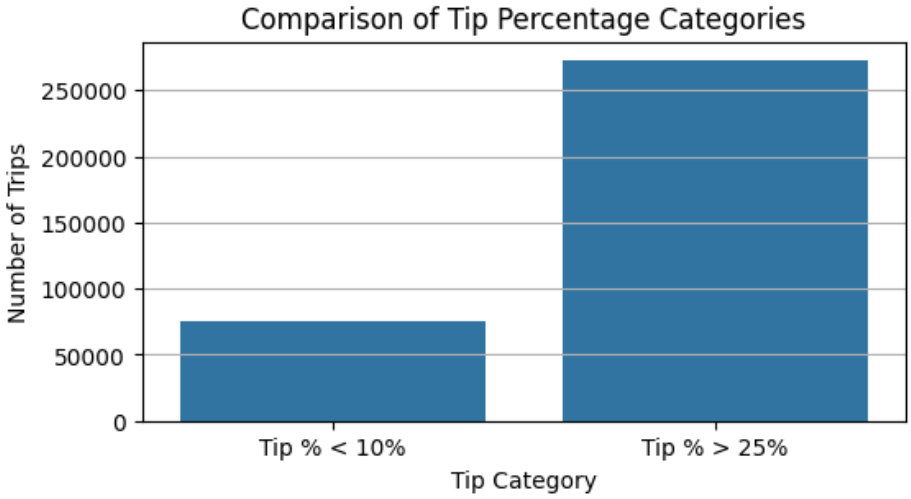


3.2.12. Compare the fare rates of different vendors in a distance-tiered fashion

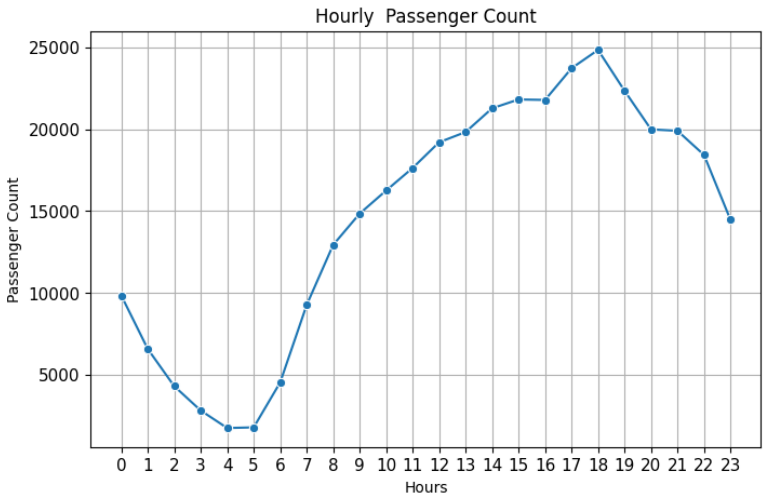


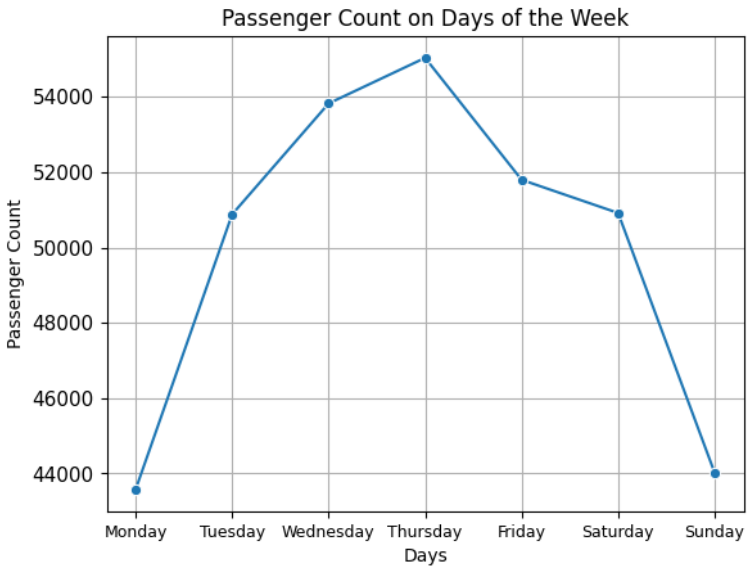
3.2.13. Analyse the tip percentages

fare_amount	tip_amount	trip_distance	passenger_count	tpep_pickup_datetime	tip_percent_by_fare	tip_percent_by_trip	tip_percent_by_passenger	tip_percent_by_pickup_time
7.9	2.58	1.24	2.0	2023-01-01 00:16:41	32.658228	208.064516	129.000000	258.0
34.5	7.90	7.10	2.0	2023-01-01 00:42:56	22.898551	111.267606	395.000000	790.0
11.4	3.28	1.59	2.0	2023-01-01 00:58:00	28.771930	206.289308	164.000000	328.0
19.1	6.02	3.16	1.0	2023-01-01 00:16:06	31.518325	190.506329	602.000000	602.0
31.7	7.09	7.64	1.0	2023-01-01 00:44:09	22.365931	92.801047	709.000000	709.0
...	...	...	...	...	...	...	...	...
15.6	4.10	2.90	1.0	2023-12-31 23:25:46	26.282051	141.379310	410.000000	410.0
70.0	8.00	21.72	1.0	2023-12-31 23:23:33	11.428571	36.832413	800.000000	800.0
4.4	1.00	0.29	2.0	2023-12-31 23:26:18	22.727273	344.827586	50.000000	100.0
9.3	2.86	1.33	1.0	2023-12-31 23:48:46	30.752688	215.037594	286.000000	286.0
70.0	14.80	18.39	3.0	2023-12-31 23:39:53	21.142857	80.478521	493.333333	1480.0



3.2.14. Analyse the trends in passenger count



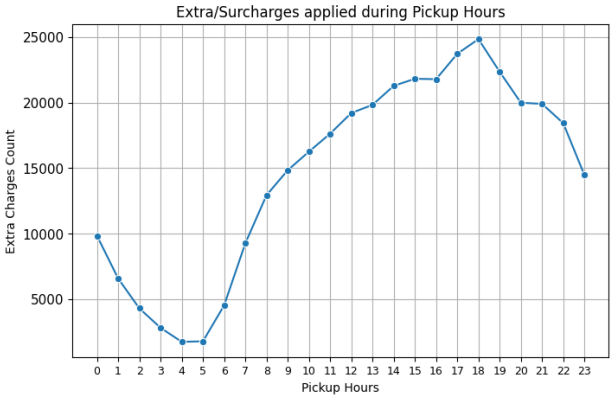


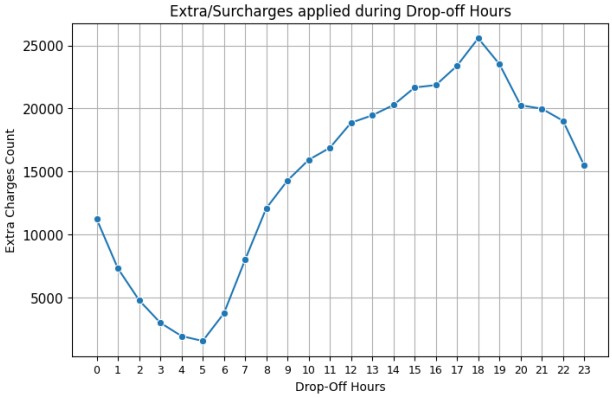
3.2.15. Analyse the variation of passenger counts across zones

zone	borough	passenger_count_by_zone
Alphabet City	Manhattan	338
Arrochar/Fort Wadsworth	Staten Island	4
Astoria	Queens	130
Auburndale	Queens	2
Baisley Park	Queens	89
...	...	...
Woodhaven	Queens	1
Woodside	Queens	52
World Trade Center	Manhattan	1838
Yorkville East	Manhattan	4531
Yorkville West	Manhattan	6797

zone	avg_passenger_count_by_zone
Alphabet City	1.426036
Arrochar/Fort Wadsworth	2.250000
Astoria	1.253846
Auburndale	1.000000
Baisley Park	1.494382
...	...
Woodhaven	1.000000
Woodside	1.480769
World Trade Center	1.523939
Yorkville East	1.311410
Yorkville West	1.342504

3.2.16. Analyse the pickup/dropoff zones or times when extra charges are applied more frequently.





From the above graph, the analysis shows that Pickup and drop-off patterns are very similar. The lowest surcharge activity occurs during the early morning hours (3 AM – 5 AM). The extra charges and surcharges are applied most frequently during the daytime and evening hours, especially between **2 PM and 7 PM**, with the peak occurring around **6PM** for both pickups and drop-offs.

4. Conclusions

4.1. Final Insights and Recommendations

4.1.1. Recommendations to optimize routing and dispatching based on demand patterns and operational inefficiencies.

Based on the complete analysis of hourly, daily, weekday/weekend, zone-level, pickup/drop-off ratios fare/tip behaviour and surcharge behaviour traffic trends, here are the key observations and recommendations:

Observation	Recommendations
Peak taxi demand occurs during evening rush hours (5 PM – 7 PM) and daytime business hours.	Increase driver deployment dynamically during peak demand periods to reduce passenger wait times and improve trip fulfillment.
Certain routes experience very slow average speeds during congestion-heavy hours.	Implement congestion-aware routing and suggest alternate faster routes instead of shortest-distance routes.
Demand falls sharply during late-night/early-morning hours (2 AM – 5 AM).	Reduce active fleet size during low-demand hours while keeping strategic standby coverage for airports and nightlife zones.

Some zones have significantly higher dropoffs than pickups, creating driver imbalance.	Introduce driver relocation incentives and intelligent rebalancing to move drivers toward high-pickup zones.
Fare per mile is highest for short trips and decreases for long-distance rides.	Revisit short-trip pricing strategy and optimize minimum fare structures to maintain profitability fairness.
Demand patterns are predictable across hours, weekdays, and zones.	Build predictive demand forecasting models for proactive fleet allocation and smarter dispatch planning.
Extra charges and congestion surcharges peak during evening traffic hours.	Use real-time congestion and surcharge analytics to dynamically optimize routing, surge pricing, and driver positioning.

4.1.2. Suggestions on strategically positioning cabs across different zones to make best use of insights uncovered by analyzing trip trends across time, days and months.

Observation	Suggestions for Strategic Cab Positioning
Peak demand occurs during evening rush hours (5 PM – 7 PM), especially in business and commercial zones.	Position more cabs near office hubs, metro stations, and commercial districts during evening peak hours to reduce passenger wait times and maximize trip utilization.
Pickup activity is highest in dense urban zones such as Manhattan and airport-connected regions.	Maintain dedicated cab pools near airports, transit hubs, hotels, and high-demand city centers for faster dispatch and improved trip coverage.
Demand drops significantly between 2 AM – 5 AM except in nightlife and airport zones.	Reduce fleet deployment in low-demand residential areas during late-night hours and strategically place standby cabs near airports and other hotspots.
Some zones experience higher dropoffs than pickups, leading to idle driver accumulation.	Use zone rebalancing strategies by redirecting idle drivers from high-dropoff zones toward nearby high-pickup demand areas using real-time analysis.
Weekend and seasonal/monthly demand patterns differ from weekday commuter traffic.	Dynamically reposition cabs based on weekend entertainment demand, tourist activity, seasonal hotspots, and monthly travel trends to improve fleet efficiency.

4.1.3. Propose data-driven adjustments to the pricing strategy to maximize revenue while maintaining competitive rates with other vendors.

Observation	Data-Driven Pricing Strategy Recommendation
Demand and surcharge activity peak during evening rush hours (5 PM – 7 PM).	Introduce dynamic peak-hour pricing during high-demand periods while ensuring prices remain within competitive market thresholds to maximize revenue.
Fare per mile is significantly higher for short-distance trips due to minimum fare structures.	Optimize minimum fare pricing for short trips by balancing base fares and congestion charges to improve profitability also ensuring affordability for frequent urban riders at the same time.
Airport and long-distance trips generate consistent demand and higher trip values.	Implement route-based smart pricing for airport and long-distance trips using real-time demand, traffic, and competitor benchmarking to maximize high-value trip revenue.
Late-night demand is lower but operational costs remain high due to lower fleet utilization.	Apply selective late-night pricing incentives or discounts in low-demand zones to improve trip volumes while avoiding excessive idle fleet costs.
Some zones consistently experience high pickup demand and low driver availability.	Use zone-level surge pricing and driver incentives in high-demand areas to improve fleet availability and reduce passenger wait times.